

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 108. Index, Sort & Reverse

Section 1: Lecture Summary

This lecture covers four important Python list methods related to searching and ordering elements: **index()**, **count()**, **reverse()**, and **sort()**. The instructor explains the parameters, default values, and behavior of each method with practical examples in a Jupyter notebook environment. Starting with the index method for finding an element's position, then counting occurrences of elements, reversing the order of list elements, and finally sorting lists in both ascending and descending order with optional customization by key functions and keyword arguments. The lecture highlights key Python concepts such as optional and mandatory parameters, keyword arguments, and sorting with custom criteria like string length or case-insensitive sorting.

Section 2: Key Concepts and Explanations

- **index(element, start=0, end=len(list))**

- Returns the index of the first occurrence of the specified element in the list.
- Parameters:
 - element (mandatory): The value to find.
 - start (optional): The index to start the search from (default 0).
 - end (optional): The index to end the search at (default end of list).
- If the element is not found in the specified range, it raises a `ValueError`.
- Useful to find specific occurrences by adjusting start and end indices.

- **count(element)**

- Returns the number of times the specified element appears in the list.
- Parameter:
 - element (mandatory): The element whose occurrences need to be counted.
- Returns 0 if the element is not present.

- **reverse()**

- Reverses the order of elements of the list in place (modifies the original list).
- Takes no arguments.
- The reversed list maintains the types and data but in opposite order.
- **sort(`key=None`, `reverse=False`)**
 - Sorts the elements of the list in place.
 - Arguments:
 - `key`: A function that extracts a comparison key from each element (optional).
 - `reverse`: Boolean, if `True` sorts in descending order; default is ascending.
 - The `` in the method signature means all arguments after it must be passed as keyword arguments.
- **Keyword Arguments in Python**
 - In methods like `sort`, parameters after `` must be specified by name (`key=`, `reverse=`).
 - This improves readability and flexibility.
- **Sorting with Custom Key Functions**
 - Using `key=len` sorts strings by their length instead of lexical order.
 - Using `key=str.lower` or `key=str.casefold` allows case-insensitive sorting, important because Python sorts uppercase before lowercase by default due to ASCII values.
- **Difference Between `reverse()` and `sort(reverse=True)`**
 - `reverse()` reverses the current list order as-is.
 - `sort(reverse=True)` sorts the list in descending order (reverse sorted).
 - They serve different use cases.

Section 3: Example Code and Use Cases

Example list with duplicate elements

`index()` method usage examples

```
L1 = [10, 20, 30, 40, 10, 20, 30, 40, 30, 20]

print(L1.index(20))          # Output: 1 (first occurrence)
print(L1.index(20, 2))       # Output: 5 (search starting from index 2)
```

```
print(L1.index(20, 2, 6))      # Output: 5 (search between indices 2 and 6)
```

Explanation: Finds index of first occurrence of 20 in specified slices of the list.

count() method to count occurrences of element 20

```
print(L1.count(20))           # Output: 3
```

Explanation: Counts how many times 20 appears in the list.

reverse() method reverses the list inplace

```
L1 = [10, 20, 30, 40, 50, 60, 70]
L1.reverse()
print(L1)                      # Output: [70, 60, 50, 40, 30, 20, 10]
```

Explanation: Reverses the order of elements in the list.

sort() method with default ascending order

sort() with reverse=True for descending order

```
L1 = [70, 10, 60, 20, 50, 30, 40]
L1.sort()
print(L1)                      # Output: [10, 20, 30, 40, 50, 60, 70]

L1 = [70, 10, 60, 20, 50, 30, 40]
L1.sort(reverse=True)
print(L1)                      # Output: [70, 60, 50, 40, 30, 20, 10]
```

Explanation: Sorts lists in ascending and descending order.

Sorting strings alphabetically

Sorting strings by length using key function

Sorting ignoring case sensitivity

Using key=str.lower to sort ignoring case

```
L1 = ['coat', 'python', 'black', 'cat']
L1.sort()
print(L1)                      # Output: ['black', 'cat', 'coat', 'python']

L1 = ['coat', 'python', 'black', 'cat']
L1.sort(key=len)
print(L1)                      # Output: ['cat', 'coat', 'black', 'python']

L1 = ['apple', 'Bat', 'cat', 'Dog']
```

```
L1.sort()
print(L1)                # Output: ['Bat', 'Dog', 'apple', 'cat']
(uppercase first)

L1.sort(key=str.lower)
print(L1)                # Output: ['apple', 'Bat', 'cat', 'Dog']
```

Explanation: Demonstrates lexical sorting, length-based sorting, and case-insensitive sorting with the `key` argument.

Section 4: Key Takeaways

- The **index()** method finds the first index of an element, with optional start and end search positions.
- The **count()** method returns how many times an element appears in a list.
- The **reverse()** method reverses the list in place without sorting.
- The **sort()** method sorts lists in place, by default in ascending order.
- Use `reverse=True` keyword arg in **sort()** for descending order.
- The `key` keyword argument in **sort()** allows sorting by custom criteria like length or case-insensitive comparison.
- Parameters after `` in function definitions require passing as keyword arguments in Python.
- Sorting strings respects ASCII order by default (uppercase < lowercase), so use `key=str.lower` to ignore case differences.
- **reverse()** and **sort(reverse=True)** have different effects: one reverses order, the other sorts in descending order.
- These list methods help efficiently manipulate and analyze list data for Python programming.